```
import os

FILE_ID = "1AC2iSqEdYVNWBibX72ctYDVJQUd6xv34"

!gdown {FILE_ID} -O /content/food.tar.gz

size_gb = os.path.getsize("/content/food.tar.gz") / 1e9
print(f"Downloaded: {size_gb:.2f} GB")

#https://drive.google.com/file/d/1AC2iSqEdYVNWBibX72ctYDVJQUd6xv34/
view?usp=drive_link
```

```
Downloading...
From (original): https://drive.google.com/uc?
id=1AC2iSqEdYVNWBibX72ctYDVJQUd6xv34
From (redirected): https://drive.google.com/uc?
id=1AC2iSqEdYVNWBibX72ctYDVJQUd6xv34&confirm=t&uuid=2724a6ef-787b-
4d2e-85bc-dd2203c0353c
To: /content/food.tar.gz
100% 5.00G/5.00G [00:53<00:00, 93.4MB/s]
Downloaded: 5.00 GB
```

```
!df -h /content | tail -1
!mkdir -p /content/data
!tar -xzf /content/food.tar.gz -C /content/data
!ls /content/data
```

```
overlay         113G   48G   66G  42% /
food-101
```

```
IMG_EXT = (".jpg", ".jpeg", ".png", ".bmp")
DATA_ROOT = "/content/data"

for root, dirs, files in os.walk(DATA_ROOT):
    dirs[:] = sorted(d for d in dirs if not d.startswith((".",
"__MACOSX")))
    depth = root[len(DATA_ROOT):].count(os.sep)
    imgs = sum(f.lower().endswith(IMG_EXT) for f in files)
    other = len(files) - imgs
    print("  " * depth + f"{os.path.basename(root) or DATA_ROOT}/  ->
{len(dirs)} subfolders, {imgs} images, {other} other files")
    if len(dirs) > 6:
        print("  " * (depth + 1) + f"(showing 3 of {len(dirs)}
subfolders: {dirs[:3]})")
        dirs[:] = dirs[:3]
```

```
data/  -> 1 subfolders, 0 images, 0 other files
  food-101/  -> 2 subfolders, 0 images, 2 other files
    images/  -> 101 subfolders, 0 images, 0 other files
      (showing 3 of 101 subfolders: ['apple_pie', 'baby_back_ribs',
'baklava'])
```

```
      apple_pie/  -> 0 subfolders, 1000 images, 0 other files
      baby_back_ribs/  -> 0 subfolders, 1000 images, 0 other files
      baklava/  -> 0 subfolders, 1000 images, 0 other files
    meta/  -> 0 subfolders, 0 images, 6 other files
```

```
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image

FOOD="/content/data/food-101"
IMAGES=FOOD+"/images"

META = FOOD + "/meta"

print(os.listdir(FOOD))
print(os.listdir(META))

classes=sorted(os.listdir(IMAGES))
print(len(classes))
print(classes)
```

```
['images', 'meta', 'README.txt', 'license_agreement.txt']
['train.txt', 'labels.txt', 'test.txt', 'classes.txt', 'test.json',
'train.json']
101
['apple_pie', 'baby_back_ribs', 'baklava', 'beef_carpaccio',
'beef_tartare', 'beet_salad', 'beignets', 'bibimbap', 'bread_pudding',
'breakfast_burrito', 'bruschetta', 'caesar_salad', 'cannoli',
'caprese_salad', 'carrot_cake', 'ceviche', 'cheese_plate',
'cheesecake', 'chicken_curry', 'chicken_quesadilla', 'chicken_wings',
'chocolate_cake', 'chocolate_mousse', 'churros', 'clam_chowder',
'club_sandwich', 'crab_cakes', 'creme_brulee', 'croque_madame',
'cup_cakes', 'deviled_eggs', 'donuts', 'dumplings', 'edamame',
'eggs_benedict', 'escargots', 'falafel', 'filet_mignon',
'fish_and_chips', 'foie_gras', 'french_fries', 'french_onion_soup',
'french_toast', 'fried_calamari', 'fried_rice', 'frozen_yogurt',
'garlic_bread', 'gnocchi', 'greek_salad', 'grilled_cheese_sandwich',
'grilled_salmon', 'guacamole', 'gyoza', 'hamburger',
'hot_and_sour_soup', 'hot_dog', 'huevos_rancheros', 'hummus',
'ice_cream', 'lasagna', 'lobster_bisque', 'lobster_roll_sandwich',
'macaroni_and_cheese', 'macarons', 'miso_soup', 'mussels', 'nachos',
'omelette', 'onion_rings', 'oysters', 'pad_thai', 'paella',
'pancakes', 'panna_cotta', 'peking_duck', 'pho', 'pizza', 'pork_chop',
'poutine', 'prime_rib', 'pulled_pork_sandwich', 'ramen', 'ravioli',
'red_velvet_cake', 'risotto', 'samosa', 'sashimi', 'scallops',
'seaweed_salad', 'shrimp_and_grits', 'spaghetti_bolognese',
'spaghetti_carbonara', 'spring_rolls', 'steak',
```

```
'strawberry_shortcake', 'sushi', 'tacos', 'takoyaki', 'tiramisu',
'tuna_tartare', 'waffles']

train_list=open(META+"/train.txt").read().split()
test_list=open(META+"/test.txt").read().split()
print(len(train_list))
print(len(test_list))
```

```
75750
25250
```

```
counts={c:len(os.listdir(IMAGES+"/"+c))for c in classes}
print(max(counts.values()))
print(min(counts.values()))
print("total", sum(counts.values()))
```

```
1000
1000
total 101000
```

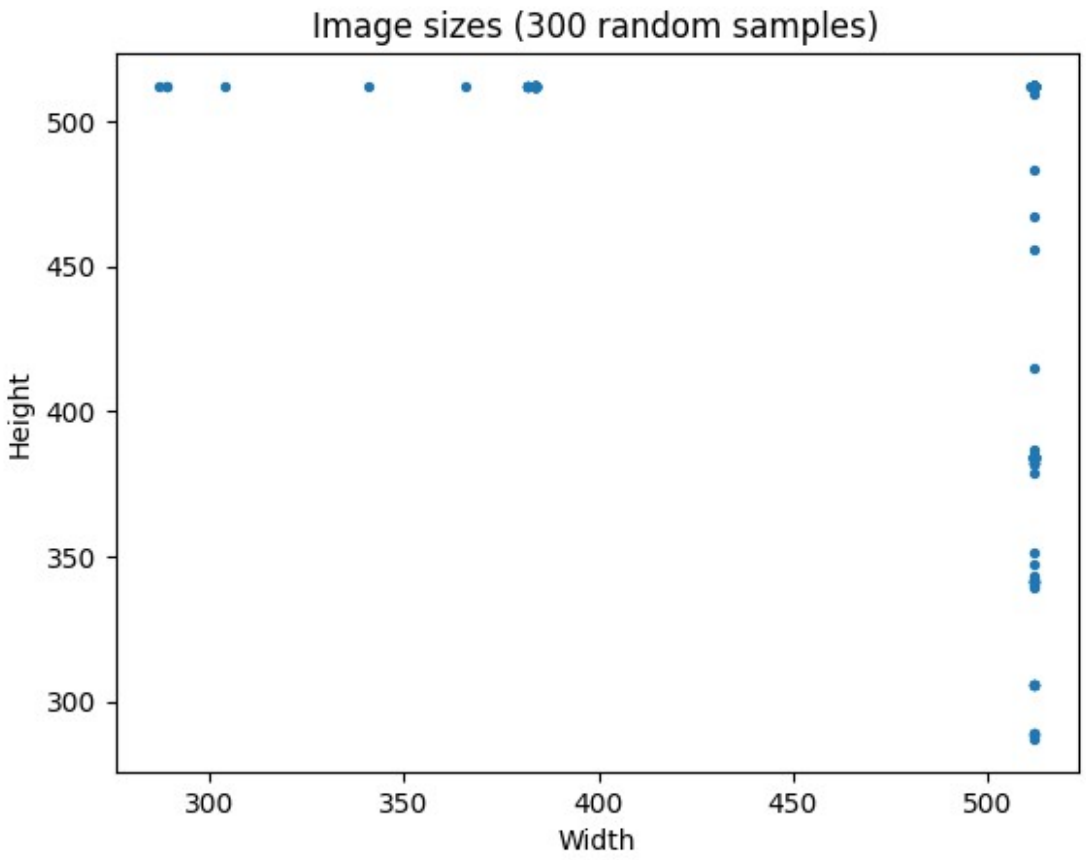
```
random.seed(42)
paths = []
for c in random.sample(classes, 20):
    files = os.listdir(IMAGES + "/" + c)
    paths += [IMAGES + "/" + c + "/" + f for f in random.sample(files,
15)]

info = []
for p in paths:
    with Image.open(p) as im:
        info.append({"w": im.size[0], "h": im.size[1], "mode":
im.mode})
info = pd.DataFrame(info)

print(info[["w", "h"]].describe().loc[["min", "mean", "max"]])
print(info["mode"].value_counts())

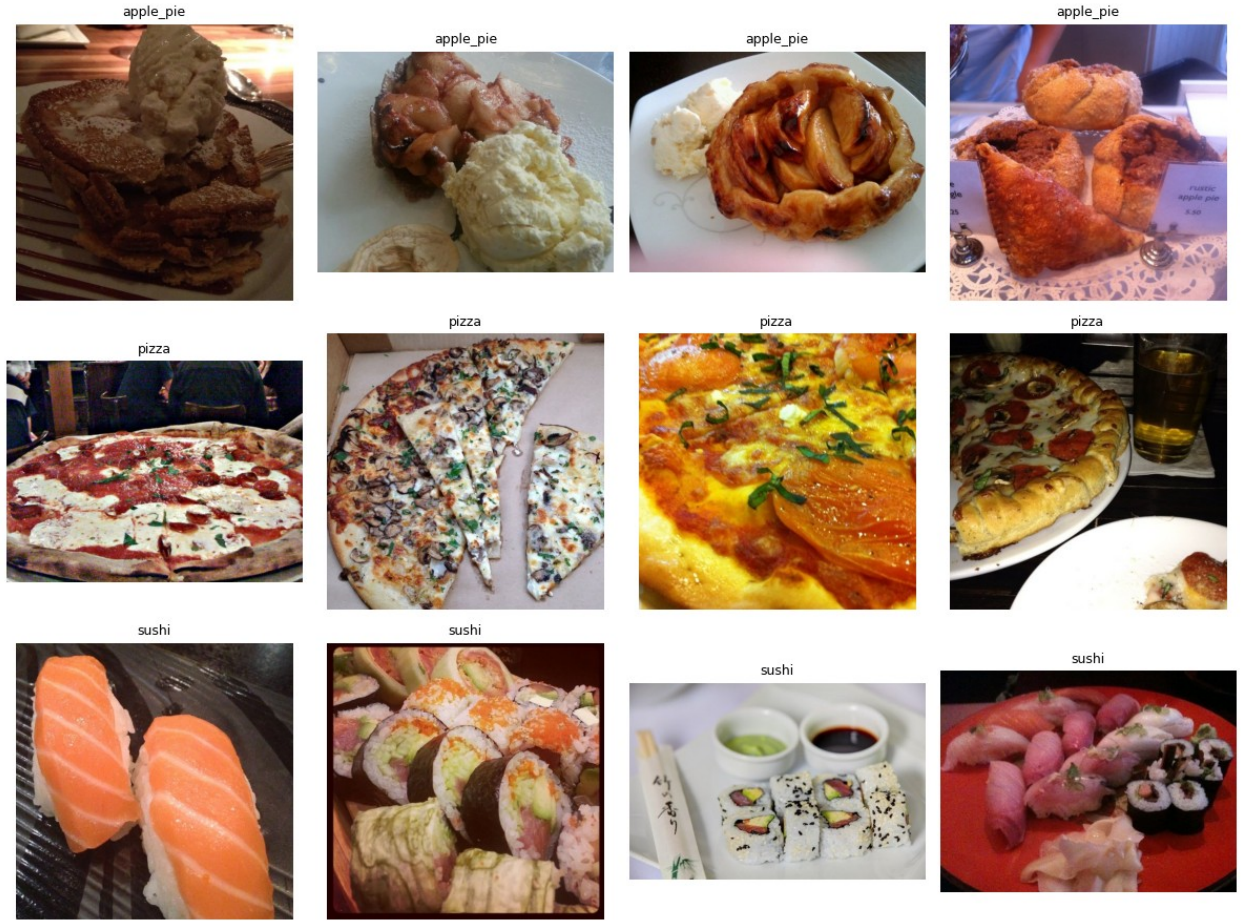
plt.scatter(info["w"], info["h"], s=8)
plt.xlabel("Width")
plt.ylabel("Height")
plt.title("Image sizes (300 random samples)")
plt.show()
```

```
                w           h
min   287.000000  287.000000
mean  492.183333  472.916667
max   512.000000  512.000000
mode
RGB    300
Name: count, dtype: int64
```




```
show_classes = ["apple_pie", "pizza", "sushi"]

fig, axes = plt.subplots(3, 4, figsize=(12, 9))
for r, c in enumerate(show_classes):
    files = random.sample(os.listdir(IMAGES + "/" + c), 4)
    for k, f in enumerate(files):
        axes[r, k].imshow(Image.open(IMAGES + "/" + c + "/" + f))
        axes[r, k].set_title(c, fontsize=9)
        axes[r, k].axis("off")
plt.tight_layout()
plt.show()
```




```
import shutil
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay

tf.random.set_seed(42)

SELECTED = ["pizza", "sushi", "hamburger", "ice_cream",
"french_fries",
            "apple_pie", "caesar_salad", "donuts", "tacos",
"pancakes"]
assert all(c in classes for c in SELECTED), "a class name is wrong"

OUT = "/content/food10"
for split_name, file_list in [("train", train_list), ("test",
test_list)]:
    for c in SELECTED:
        os.makedirs(f"{OUT}/{split_name}/{c}", exist_ok=True)
```

```
    for item in file_list:                          # each item looks
like "pizza/12345"
        c, name = item.split("/")
        if c in SELECTED:
            shutil.copy(f"{IMAGES}/{c}/{name}.jpg",
f"{OUT}/{split_name}/{c}/{name}.jpg")

TRAIN_DIR = OUT + "/train"
TEST_DIR = OUT + "/test"
print("train images:", sum(len(os.listdir(TRAIN_DIR + "/" + c)) for c
in SELECTED))
print("test images: ", sum(len(os.listdir(TEST_DIR + "/" + c)) for c
in SELECTED))
print("GPU:", tf.config.list_physical_devices("GPU"))
```

```
train images: 7500
test images:  2500
GPU: [PhysicalDevice(name='/physical_device:GPU:0',
device_type='GPU')]
```

```
IMG_SIZE=(128,128)
BATCH=32

train_ds = keras.utils.image_dataset_from_directory(
    TRAIN_DIR, label_mode="categorical", image_size=IMG_SIZE,
batch_size=BATCH,
    validation_split=0.2, subset="training", seed=42)

val_ds = keras.utils.image_dataset_from_directory(
    TRAIN_DIR, label_mode="categorical", image_size=IMG_SIZE,
batch_size=BATCH,
    validation_split=0.2, subset="validation", seed=42)

test_ds = keras.utils.image_dataset_from_directory(
    TEST_DIR, label_mode="categorical", image_size=IMG_SIZE,
batch_size=BATCH, shuffle=False)

class_names = train_ds.class_names
print(class_names)
```

```
Found 7500 files belonging to 10 classes.
Using 6000 files for training.
Found 7500 files belonging to 10 classes.
Using 1500 files for validation.
Found 2500 files belonging to 10 classes.
['apple_pie', 'caesar_salad', 'donuts', 'french_fries', 'hamburger',
'ice_cream', 'pancakes', 'pizza', 'sushi', 'tacos']
```

```
reg = keras.regularizers.l2(1e-4)

model = keras.Sequential([
```

```
    layers.Input(shape=(128, 128, 3)),
    layers.Rescaling(1.0 / 255),

    # augmentation (active only during training)
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.15),
    layers.RandomZoom(0.2),
    layers.RandomTranslation(0.1, 0.1),
    layers.RandomContrast(0.2),

    layers.Conv2D(32, (3, 3), padding="same", activation="relu",
kernel_regularizer=reg),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.2),

    layers.Conv2D(64, (3, 3), padding="same", activation="relu",
kernel_regularizer=reg),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.2),

    layers.Conv2D(128, (3, 3), padding="same", activation="relu",
kernel_regularizer=reg),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.3),

    layers.Conv2D(256, (3, 3), padding="same", activation="relu",
kernel_regularizer=reg),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),

    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.5),
    layers.Dense(10, activation="softmax"),
])
model.summary()
```

```
Model: "sequential_2"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━┩
┃ rescaling_2 (Rescaling)              ┃ (None, 128, 128, 3)         ┃
0 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━┫
```

```
──────┤
| random_flip_2 (RandomFlip)       | (None, 128, 128, 3)     |
0 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| random_rotation_2                | (None, 128, 128, 3)     |
0 |
| (RandomRotation)                 |                         |
|
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| random_zoom_2 (RandomZoom)       | (None, 128, 128, 3)     |
0 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| random_translation               | (None, 128, 128, 3)     |
0 |
| (RandomTranslation)              |                         |
|
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| random_contrast                  | (None, 128, 128, 3)     |
0 |
| (RandomContrast)                 |                         |
|
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| conv2d_8 (Conv2D)                | (None, 128, 128, 32)    |
896 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| batch_normalization_8            | (None, 128, 128, 32)    |
128 |
| (BatchNormalization)             |                         |
|
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| max_pooling2d_8 (MaxPooling2D)   | (None, 64, 64, 32)      |
0 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| dropout_5 (Dropout)              | (None, 64, 64, 32)      |
0 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
| conv2d_9 (Conv2D)                | (None, 64, 64, 64)      |
18,496 |
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
```

```
| batch_normalization_9            | (None, 64, 64, 64)     |
256 |
| (BatchNormalization)             |                        |
|
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
| max_pooling2d_9 (MaxPooling2D)   | (None, 32, 32, 64)     |
0 |
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
| dropout_6 (Dropout)              | (None, 32, 32, 64)     |
0 |
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
| conv2d_10 (Conv2D)               | (None, 32, 32, 128)    |
73,856 |
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
| batch_normalization_10           | (None, 32, 32, 128)    |
512 |
| (BatchNormalization)             |                        |
|
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
| max_pooling2d_10 (MaxPooling2D)  | (None, 16, 16, 128)    |
0 |
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
| dropout_7 (Dropout)              | (None, 16, 16, 128)    |
0 |
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
| conv2d_11 (Conv2D)               | (None, 16, 16, 256)    |
295,168 |
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
| batch_normalization_11           | (None, 16, 16, 256)    |
1,024 |
| (BatchNormalization)             |                        |
|
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
| max_pooling2d_11 (MaxPooling2D)  | (None, 8, 8, 256)      |
0 |
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
| global_average_pooling2d_2       | (None, 256)            |
0 |
| (GlobalAveragePooling2D)         |                        |
```

```
┃                                                                 ┃
┣━━━
```

```
1.4659 - val_accuracy: 0.4253 - val_loss: 2.0404 - learning_rate:
2.5000e-04
Epoch 6/20
188/188 ━━━━━━━━━━━━━━━━━━━━ 13s 68ms/step - accuracy: 0.6323 - loss:
1.4442 - val_accuracy: 0.3547 - val_loss: 2.4431 - learning_rate:
2.5000e-04
Epoch 7/20
188/188 ━━━━━━━━━━━━━━━━━━━━ 20s 67ms/step - accuracy: 0.6263 - loss:
1.4531 - val_accuracy: 0.4467 - val_loss: 1.9221 - learning_rate:
2.5000e-04
Epoch 8/20
188/188 ━━━━━━━━━━━━━━━━━━━━ 13s 71ms/step - accuracy: 0.6368 - loss:
1.4312 - val_accuracy: 0.3607 - val_loss: 2.3141 - learning_rate:
2.5000e-04
Epoch 9/20
188/188 ━━━━━━━━━━━━━━━━━━━━ 13s 67ms/step - accuracy: 0.6347 - loss:
1.4304 - val_accuracy: 0.4600 - val_loss: 1.9233 - learning_rate:
2.5000e-04
Epoch 10/20
188/188 ━━━━━━━━━━━━━━━━━━━━ 13s 68ms/step - accuracy: 0.6333 - loss:
1.4197 - val_accuracy: 0.4333 - val_loss: 2.0305 - learning_rate:
2.5000e-04
Epoch 11/20
188/188 ━━━━━━━━━━━━━━━━━━━━ 13s 67ms/step - accuracy: 0.6567 - loss:
1.3939 - val_accuracy: 0.4160 - val_loss: 2.0623 - learning_rate:
1.2500e-04
Epoch 12/20
188/188 ━━━━━━━━━━━━━━━━━━━━ 13s 67ms/step - accuracy: 0.6565 - loss:
1.3892 - val_accuracy: 0.4200 - val_loss: 2.0392 - learning_rate:
1.2500e-04
Epoch 13/20
188/188 ━━━━━━━━━━━━━━━━━━━━ 13s 67ms/step - accuracy: 0.6508 - loss:
1.3971 - val_accuracy: 0.4173 - val_loss: 2.0395 - learning_rate:
1.2500e-04
Epoch 14/20
188/188 ━━━━━━━━━━━━━━━━━━━━ 13s 67ms/step - accuracy: 0.6657 - loss:
1.3753 - val_accuracy: 0.4387 - val_loss: 2.0379 - learning_rate:
6.2500e-05
Epoch 15/20
188/188 ━━━━━━━━━━━━━━━━━━━━ 21s 67ms/step - accuracy: 0.6630 - loss:
1.3679 - val_accuracy: 0.4680 - val_loss: 1.9441 - learning_rate:
6.2500e-05

test_loss, test_acc = model.evaluate(test_ds)
print("Test loss:", test_loss)
print("Test accuracy:", test_acc)

y_true = np.concatenate([y.numpy() for x, y in
test_ds]).argmax(axis=1)
y_pred = model.predict(test_ds).argmax(axis=1)
```

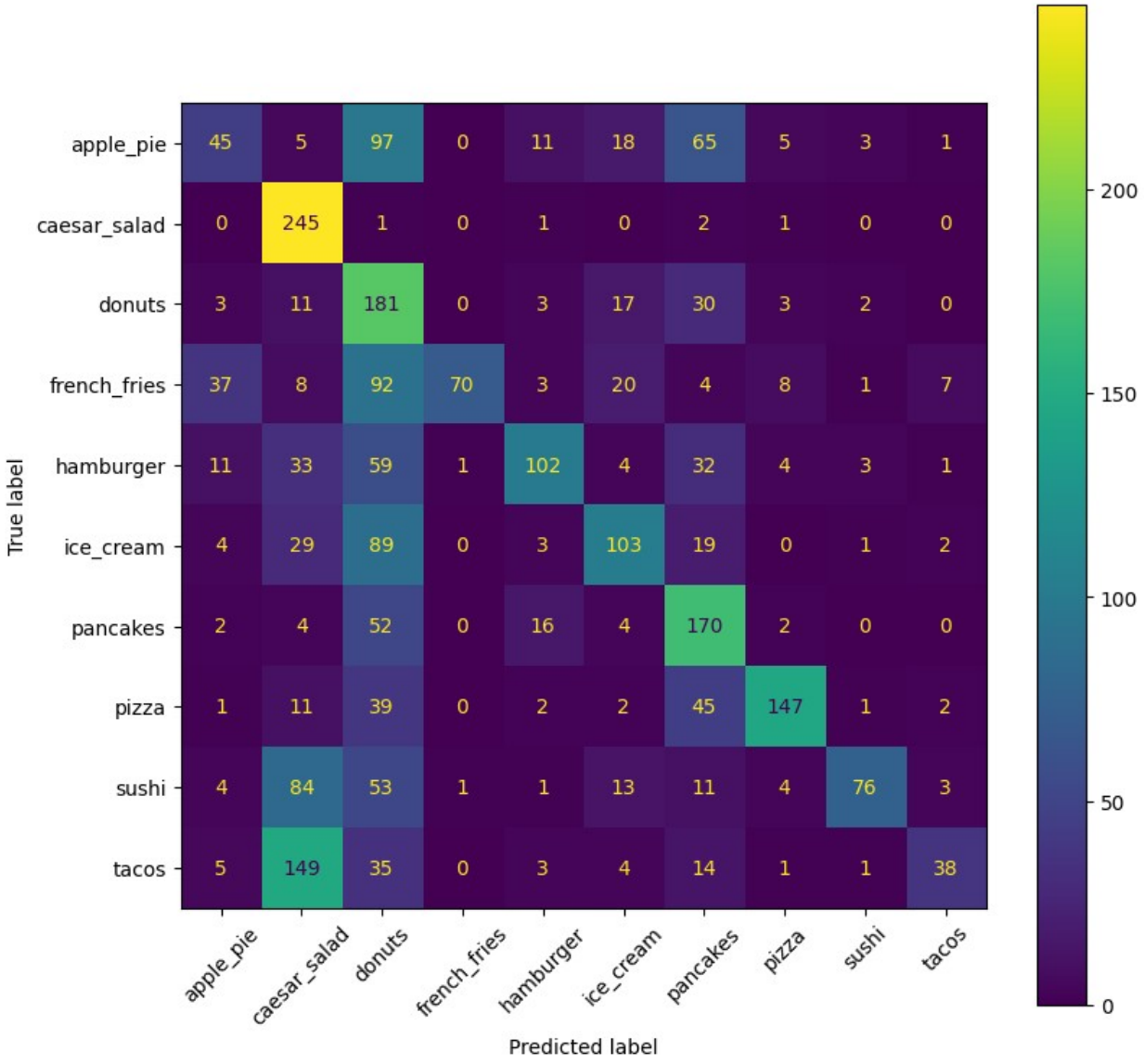
```
print(classification_report(y_true, y_pred, target_names=class_names))

fig, ax = plt.subplots(figsize=(9, 9))
ConfusionMatrixDisplay.from_predictions(y_true, y_pred,
display_labels=class_names,
                                        xticks_rotation=45, ax=ax)
plt.show()
```

```
79/79 ━━━━━━━━━━━━━━━━━━━━ 3s 40ms/step - accuracy: 0.4708 - loss:
1.8322
Test loss: 1.8322288990020752
Test accuracy: 0.4708000123500824
79/79 ━━━━━━━━━━━━━━━━━━━━ 3s 41ms/step
              precision    recall  f1-score   support

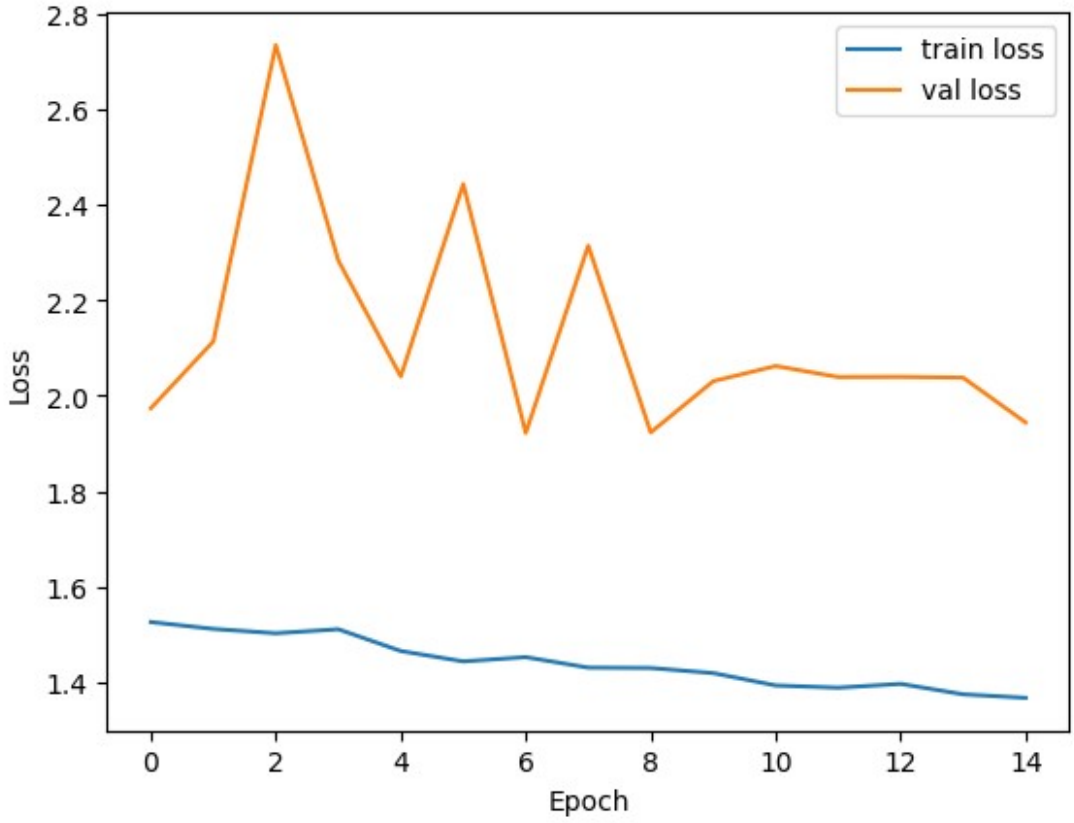
   apple_pie       0.40      0.18      0.25       250
caesar_salad       0.42      0.98      0.59       250
      donuts       0.26      0.72      0.38       250
french_fries       0.97      0.28      0.43       250
   hamburger       0.70      0.41      0.52       250
   ice_cream       0.56      0.41      0.47       250
    pancakes       0.43      0.68      0.53       250
       pizza       0.84      0.59      0.69       250
       sushi       0.86      0.30      0.45       250
       tacos       0.70      0.15      0.25       250

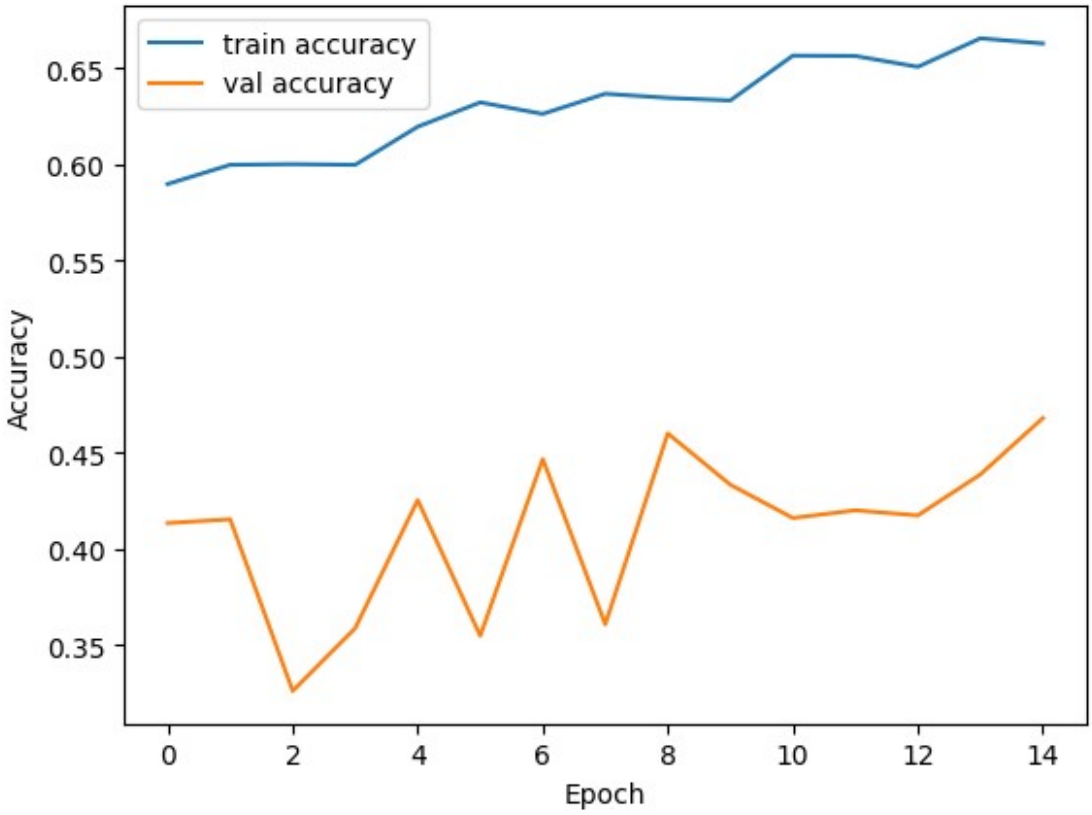
    accuracy                           0.47      2500
   macro avg       0.62      0.47      0.46      2500
weighted avg       0.62      0.47      0.46      2500
```




```
plt.plot(history.history["loss"], label="train loss")
plt.plot(history.history["val_loss"], label="val loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

plt.plot(history.history["accuracy"], label="train accuracy")
plt.plot(history.history["val_accuracy"], label="val accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```


train loss
val loss
Loss
Epoch
2.8
2.6
2.4
2.2
2.0
1.8
1.6
1.4
0
2
4
6
8
10
12
14


train accuracy
val accuracy
Accuracy
Epoch
0.65
0.60
0.55
0.50
0.45
0.40
0.35
0
2
4
6
8
10
12
14